Microservice Architecture (/index.html)

Supported by Kong (https://konghq.com/)

Pattern: 3rd Party Registration

🏷 pattern (/tags/pattern) 🏷 inter-service communication (/tags/inter-service communication) 🏷 service discovery (/tags/service discovery)

Context

You have applied either the Client-side Service Discovery pattern (client-side-discovery.html) or the Server-side Service Discovery pattern (server-side-discovery.html). Service instances must be registered with the service registry (service-registry.html) on startup so that they can be discovered and unregistered on shutdown.

Problem

How are service instances registered with and unregistered from the service registry?

Forces

- Service instances must be registered with the service registry on startup and unregistered on shutdown
- Service instances that crash must be unregistered from the service registry
- Service instances that are running but incapable of handling requests must be unregistered from the service registry

Solution

A 3rd party registrar is responsible for registering and unregistering a service instance with the service registry. When the service instance starts up, the registrar registers the service instance with the service registry. When the service instance shuts downs, the registrar unregisters the service instance from the service registry.

Examples

Examples of the 3rd Party Registration pattern include:

- Netflix Prana (https://github.com/Netflix/Prana) - a “side car” application that runs along side a non-JVM application and registers the application with Eureka.
- AWS Autoscaling Groups (http://aws.amazon.com/autoscaling/) automatically (un)registers EC2 instances with Elastic Load Balancer
- Joyent’s Container buddy (https://github.com/joyent/containerbuddy) runs in a Docker container as the parent process for the service and registers it with the registry
- Registrator (https://github.com/gliderlabs/registrator) - registers and unregisters Docker containers with various service registries
- Clustering frameworks such as Kubernetes (https://github.com/GoogleCloudPlatform/kubernetes/blob/master/docs/services.md) and Marathon (https://mesosphere.github.io/marathon/docs/service-discovery-load-balancing.html) (un)register service instances with the built-in/implicit registry

Resulting context

The benefits of the 3rd Party Registration pattern include:

- The service code is less complex than when using the Self Registration pattern (self-registration.html) since its not responsible for registering itself
- The registrar can perform health checks on a service instance and register/unregister the instance based the health check

There are also some drawbacks:

- The 3rd party registrar might only have superficial knowledge of the state of the service instance, e.g. RUNNING or NOT RUNNING and so might not know whether it can handle requests. However, as mentioned above some registrars such as Netflix Prana perform a health check in order to determine the availability of the service instance.
- Unless the registrar is part of the infrastructure it's another component that must be installed, configured and maintained. Also, since it's a critical system component it needs to be highly available.

Related patterns

- Service Registry (service-registry.html)
- Client Side Discovery (client-side-discovery.html)
- Server Side Discovery (server-side-discovery.html)
- Self Registration (self-registration.html) is an alternative solution

---

🏷 pattern (/tags/pattern) 🏷 inter-service communication (/tags/inter-service communication) 🏷 service discovery (/tags/service discovery)

Modernization Help (https://chrisrichardson.net/)

Copyright © 2026 Chris Richardson • All rights reserved • Supported by Kong (https://konghq.com/).

ALSO ON MICROSERVICES

How modular can your monolith go? Part 5 - ...

2 years ago · 2 comments

How modular can your monolith go? Part 5 - decoupling domains with ...

Microservices adoption anti-patterns

3 years ago · 1 comment

Microservices adoption anti-patterns: microservices are a magic pixie dust Many ...

Beer and tacos and the metaphorical origins ...

3 years ago · 1 comment

Beer and tacos and the metaphorical origins of Docker containers ...

Redu in a n

a year

Reduc micros During

1 Comment

Login

G

Join the discussion...

LOG IN WITH

OR SIGN UP WITH DISQUS

Name

1 Share

Best Newest Oldest

E

ejlp12

9 years ago

Joyent's Containerbuddy is now ContainerPilot

https://www.joyent.com/blog...

0 0 Reply

Subscribe Privacy Do Not Sell My Data